TEAM 67 · COMPUTER NETWORKS · PHASE 1

Private Network Service Platform

From a private name to an HTTPS response, served by one of two backends, with every step captured on the wire.

Tathagat Harsh (2401010477) · **MS Thejas** (2401010255) · **Garv Verma** (2401010168)

app.team67.test · DNS → TCP → TLS → HTTP · three MacBooks, one LAN, no cloud

DEMO PLAN · SPEC §8 ORDER

Eight steps, each one a layer we can prove

#	What we show	Presenter
1	Topology, roles, IP and service table	Tathagat (each states own role)
2	All three Macs on one LAN	Everyone
3	Resolve app.team67.test through our DNS	Garv Verma · Tathagat
4	HTTPS by name, no certificate warnings	MS Thejas · Garv Verma
5	Load balancing across Backend A and B	Tathagat · Garv Verma
6	Wireshark: DNS, TCP and TLS on the wire	Garv Verma
7	HTTP headers and caching (304)	MS Thejas
8	Backend failure, service continues, recovery	MS Thejas · Garv Verma

TEAM AND ROLES

Three people, the spec's four network roles

NODE 1

Tathagat Harsh

Enrollment 2401010477

Spec Mac 1 + Mac 2: DNS and edge

- dnsmasq, DNS on port 53
- nginx reverse proxy and load balancer
- TLS termination, team CA
- Integration and documentation

Presents: architecture, DNS, load balancing

NODE 2

MS Thejas

Enrollment 2401010255

Spec Mac 3: Backend A + client

- Python backend on port 3001
- Endpoint and backend testing
- Browser client (HTTPS, padlock)
- Caching demo

Presents: HTTPS in the browser, caching, failover

NODE 3

Garv Verma

Enrollment 2401010168

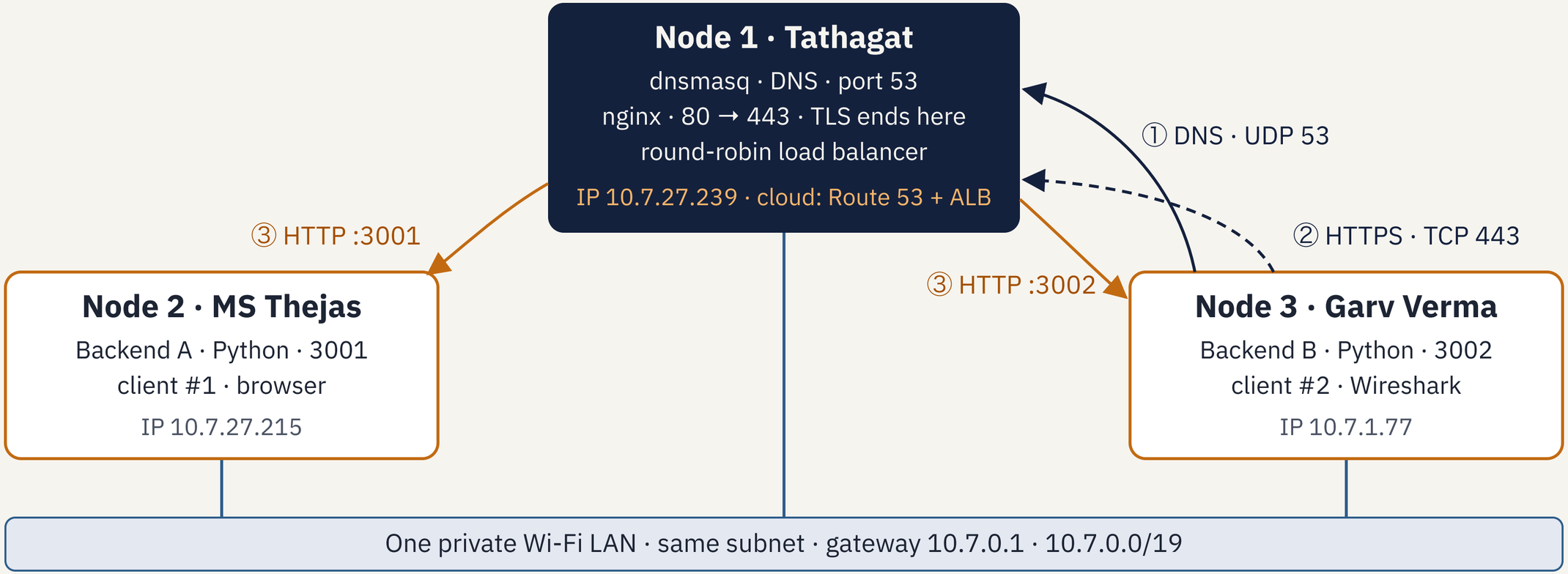
Spec Mac 4: Backend B + client

- Python backend on port 3002
- curl client
- Wireshark packet analysis
- Failure and recovery tests

Presents: dig, A/B loop, Wireshark, failures

TOPOLOGY

One LAN, three Macs, one entry point



Clients only ever know app.team67.test · only nginx knows the backends · Team 67 · 4

IP AND SERVICE INVENTORY

Every address, port and record in one table

Node	Member	IPv4 / prefix	Interface · MAC	Services (port)
Node 1	Tathagat	10.7.27.239 / 19	en0 · 7e:54:32:73:01:15	dnsmasq 53, nginx 80 → 443
Node 2	MS Thejas	10.7.27.215 / 19	en0 · 6e:f1:6c:21:19:6a	Backend A 3001
Node 3	Garv Verma	10.7.1.77 / 19	en0 · a2:dc:7b:8f:5e:71	Backend B 3002

DNS records

`app.team67.test → 10.7.27.239`
`api.team67.test → 10.7.27.239`
`TTL 60 s · zone answered locally`

Gateway

Default gateway 10.7.0.1 (mask 255.255.224.0) for all three. Same subnet, so traffic between the Macs never passes a router.

Values copied from evidence/A-lan, captured live on each Mac · Team 67 · 5

ONE REQUEST, LAYER BY LAYER

curl https://app.team67.test/api/status

1 · DNS · UDP 53

Name to IP

Client asks Node 1. dnsmasq owns team67.test and answers Node 1's IP, TTL 60.

2 · TCP · port 443

Three-way handshake

SYN, SYN-ACK, ACK from an ephemeral port. Frame goes straight to Node 1's MAC.

3 · TLS

Handshake and trust

SNI app.team67.test, certificate chains to Team67 Local Root CA, ALPN picks h2.

4 · HTTP/2 in TLS

Encrypted request

GET /api/status travels as TLS Application Data; Wireshark sees only ciphertext.

5 · nginx

Terminate and balance

Decrypts, picks the next backend round-robin, opens plain HTTP to 3001 or 3002.

6 · Backend

Answer and identify

JSON plus X-Backend: A or B. The backend sees only the edge's IP, never the client's.

PROTOCOLS AND LAYERS

Where each protocol sits, and how we show it

Protocol	OSI	TCP/IP	Shown with
HTTP/1.1, HTTP/2 · DNS	7 Application	Application	curl -v, browser DevTools, dig, nslookup
TLS 1.2 / 1.3	5–6 Session / Presentation	above Transport	Wireshark tls.handshake, curl -v
TCP (443, 3001, 3002) · UDP (53)	4 Transport	Transport	SYN / SYN-ACK / ACK, ports, seq / ack
IPv4	3 Network	Internet	ipconfig, ping (TTL 64)
Wi-Fi / Ethernet, ARP	2 Data link	Link	MAC addresses in the capture

HTTP/3 (QUIC over UDP 443) and email protocols are explanation-only in this project.

DEMO · STEPS 2–3 · EVERYONE, THEN GARV + TATHAGAT

LAN and private DNS

WE RUN

```
ping -c 3 10.7.27.215
```

```
dig app.team67.test
```

```
nslookup app.team67.test
```

Tathagat keeps the dnsmasq tab visible: the query prints live.

LOOK FOR

- 0% loss, TTL 64: same subnet
- ANSWER: app.team67.test → 10.7.27.239, TTL 60
- SERVER: 10.7.27.239#53: our DNS answered
- dnsmasq log: query from 10.7.1.77 and 10.7.27.215

DEMO · STEP 4 · THEJAS + GARV

HTTPS by name, no warnings

WE RUN

```
curl -i https://app.team67.test/api/status
```

Browser: **https://app.team67.test** → padlock → certificate

TLS 1.2 HANDSHAKE

ClientHello (SNI, ALPN) → ServerHello + Certificate + KeyExchange → ClientKeyExchange + ChangeCipherSpec + Finished → server Finished

LOOK FOR

- HTTP/2 200, no -k, no IP in the URL
- Issued to app.team67.test
- Issued by Team67 Local Root CA
- CA trusted in each client's System keychain

DEMO · STEP 5 · TATHAGAT + GARV

Round-robin across two backends

WE RUN

```
upstream team67_backends {
  server 10.7.27.215:3001 max_fails=1
fail_timeout=10s;
  server 10.7.1.77:3002  max_fails=1
fail_timeout=10s;
}
```

```
for i in 1 2 3 4; do
  curl -si https://app.team67.test/api/status | grep
-i x-backend
done
```

LOOK FOR

- X-Backend alternates A, B, A, B
- Backend logs: from=10.7.27.239
- Access log: upstream= alternating
- Client never sees a backend IP

DEMO · STEP 6 · GARV

The whole request on the wire

Packet	Layer	What we point at	Wireshark filter
3–4	DNS / UDP	Query and answer 10.7.27.239, port 65190 → 53	dns.qry.name == "app.team67.test"
5–7	TCP	SYN, SYN-ACK, ACK · port 57006 → 443	tcp.flags.syn == 1
8, 11, 13, 15	TLS 1.2	ClientHello, ServerHello, Certificate, ChangeCipherSpec	tls.handshake
16	TLS data	HTTP/2 encrypted: unreadable on the wire	tls.app_data
8	Ethernet	Dst MAC 7e:54:32:73:01:15 = Node 1's own, not the gateway's	frame details

Captured on Garv Verma's Mac with curl --tls-max 1.2, so the Certificate message stays visible (TLS 1.3 encrypts it).

DEMO · STEP 7 · THEJAS

Cache headers and a 304

WE RUN

```
curl -I https://app.team67.test/api/cached
```

```
curl -I -H "If-None-Match: [ETag]" \
  https://app.team67.test/api/cached
```

DevTools → Network: second fetch('/api/cached') shows **(disk cache)**

LOOK FOR

- cache-control: public, max-age=60
- etag: same value from A and B
- HTTP/2 304, no body
- Fresh hit: no request at all

DEMO · STEP 8 + SPEC §6.3 · THEJAS, GARV, TATHAGAT

Break one layer at a time

We break	Expected observation	Proves
Backend A stopped (Ctrl+C)	Only B answers; nginx error log: connect() failed	Edge fails over
Backend A restarted	After 10 s, A and B alternate again	Recovery
Both backends stopped	DNS and TLS still work; nginx returns 502	Where the edge ends
Client uses 8.8.8.8 for DNS	NXDOMAIN, yet ping to Node 1 works	DNS ≠ IP reachability
Record points to MS Thejas's IP	Name resolves; TCP 443 refused	DNS is a directory
Port 8443 instead of 443	Host reachable, connection refused	IP picks host, port picks service

Bold rows are the live demo (step 8); the rest on request · Team 67 · 13

DESIGN DECISIONS

What we chose, and why

Decision	Reason
Ports 443 and 80 (80 only redirects)	Real HTTPS port, no :8443 in URLs; Node 1 needs root for DNS anyway
Own root CA, SAN names only	Clients trust once; no IP in the certificate forces name-based access
TLS 1.2 and 1.3 enabled	1.3 by default; 1.2 for the capture so the certificate is visible
Round-robin, passive health checks	Visible A/B alternation; max_fails=1, fail_timeout=10s, retry on error
Python standard library backend	Nothing to install; one file, BACKEND and PORT pick A or B; binds 0.0.0.0
.test domain, dnsmasq	.test is reserved for testing; .local would clash with macOS mDNS

SINGLE POINT OF FAILURE

Node 1 holds both DNS and the edge.

If Tathagat's Mac stops, names stop resolving and HTTPS stops. Backend failure is already survived. Phase 2 removes this: a backup DNS resolver on another Mac, and a standby nginx with a DNS-based cutover.

HOW WE DIAGNOSE ANY FAULT

Bottom-up, one layer at a time

1 · DNS	2 · Network	3 · TCP	4 · TLS	5 · App
`dig`	`ping`	`nc -vz 443`	`curl -v`	`status, X-Backend`

Thank you. Questions, and the individual viva.

Tathagat Harsh 2401010477 · MS Thejas 2401010255 · Garv Verma 2401010168 · Team 67 · 16